

# CG-HIK: Query-Conditioned Computation Allocation for Verified Online Inverse Kinematics

Anonymous Author(s)

Affiliation withheld for blinded review

## Abstract

Online inverse kinematics (IK) must generate valid joint commands for queries with different numerical demands. Fixed cascades already stop after an early success, but their entry point can still impose unnecessary refinement prefixes or spend large budgets on unsuccessful requests. We investigate the hypothesis that robust coverage need not require uniform worst-case computation. CG-HIK learns query-conditioned pathway costs from action-complete observations of a shared solver portfolio and selects an entry using predicted tail latency. Learning allocates solver effort; numerical geometry generates joint commands; deterministic verification governs acceptance.

We evaluate the frozen method through the original held-out tests and a supplementary mechanism study on Panda and UR5e. On 2,500 witness-feasible point queries per robot, CG-HIK matched always-hard success but increased mean end-to-end time by 3.33% and 29.32%; a geometry rule and a reject-only control performed similarly. Selecting by predicted P95 rather than P50 reduced FEV by 0.58% and 1.86%, without a resolved P95 or P99 advantage. Reanalysis showed that 99.07% and 101.31% of the original trajectory time savings lay in trajectories neither method completed, offsetting additional cost on jointly completed paths. On new continuously feasible reference paths, CG-HIK completed 34/40 versus 37/40 for always-hard on Panda and 36/40 for both on UR5e. A Panda joint-limit family retained successful-solve savings, whereas near-singular branch evolution caused three lost completions. Official TRAC-IK provided a substantially faster external comparison. These findings identify failure handling as the dominant aggregate benefit in the tested portfolio and show why learned entry selection, successful-path computation, and acceptance validity require separate evidence.

**Keywords:** inverse kinematics; adaptive computation; algorithm selection; tail latency; damped least squares; hybrid learning and optimization; deterministic verification

## 1 Introduction

Online inverse kinematics (IK) repeatedly converts end-effector targets into joint commands. A request consists of a desired pose, the previous accepted configuration, and a finite command interval. The returned command changes the state from which the next request is solved, making branch continuity and recovery part of the problem rather than optional post-processing [1–4]. Nearby targets may need different amounts of work because the current configuration can approach a singularity, a joint limit, or an unfavorable inverse branch. Online IK therefore couples query-level computational cost to the trajectory of accepted commands.

Analytical and numerical IK supply complementary tools for this interface. Structural decompositions can enumerate branches rapidly for compatible geometries [5–7]. Jacobian and damped-least-squares (DLS) methods exploit a good local initialization, with damping limiting amplification near rank loss [8–10]. Restarts, constrained optimization, and complementary solver portfolios broaden recovery [11–15]. Their common strength is explicit geometry: a proposed configuration can be checked against pose, joint, and single-step continuity conditions before acceptance. The availability of several useful stages, however, leaves their scheduling open.

Learned and generative IK improve a different part of the pipeline: finding useful candidates in a set-valued inverse map [16]. Conditional flows, diffusion, graph representations, invertible networks, and set predictors represent alternative solutions or redundancy [17–22]. History conditioning can favor the neighborhood of a previous configuration [3, 4, 23]. These models amortize proposal generation, but candidate quality and verified system cost are different quantities. Similar proposal errors can lead to different refinement work, and a neural output can still violate a command constraint.

Hybrid proposal-refinement systems combine these strengths. Learned candidates supply an initialization; numerical geometry restores precision; explicit checks govern the returned command. IKFlow and CycleIK support numerical refinement [17, 24], while path-level generative proposals can be coupled to optimization and checking [2]. Diffusion-seeded, execution-aware, graph, and neuro-fuzzy approaches offer further forms of

this cooperation [25–29]. Once several refinement paths coexist, a hybrid system also has to decide how much of that available computation to use for the current query.

A fixed cascade already implements conditional computation: it stops on verified success and escalates on failure. Its remaining limitation is the fixed *entry*, not execution of every stage on every request. An inexpensive prefix can either solve a continuation or add work before a later stage that was sufficient on its own. Requests with little prospect of success can consume a large fallback budget. Seed selection and failure detection address related decisions [30, 31], but do not by themselves establish that learning complete pathway costs improves on a strong fixed entry or a simple geometric rule. The allocation question must include prediction overhead and distinguish savings during successful solving from savings in failure handling.

We study this question with CG-HIK, a query-conditioned allocation policy over one shared numerical portfolio, following the per-instance algorithm-selection viewpoint [32, 33]. The hypothesis is that *robust coverage does not require uniform worst-case computation*. Executing every entry on the same development queries replaces manual difficulty labels with observed success and cost. The policy predicts pathway latency, selects a sufficient entry, and can decline a request predicted to exhaust the portfolio. Its responsibilities remain separate: *learning allocates solver effort per query; numerical geometry generates joint commands; deterministic verification governs acceptance*. The frozen implementation still prepares learned candidates on every query; its allocation decision concerns subsequent numerical work, not elimination of all neural inference on easy requests.

Our contributions are the shared-contract allocation formulation, action-complete pathway supervision, and an empirical separation of entry selection, rejection, prediction overhead, and closed-loop effects. The original held-out tests compare the method with a fixed early-entry cascade. A supplementary study then asks the stronger questions that this comparison leaves open: whether the learned entry beats hard entry or a simple rule, whether P95 selection adds measurable value, whether savings persist on continuously feasible paths, and where the system sits relative to an external numerical solver. The resulting evidence includes local successful-solve gains and substantial failure-handling savings, but does not support a general learned-routing advantage over always-hard. This distinction makes the resource-allocation hypothesis testable rather than assuming that a lower aggregate runtime identifies its mechanism.

## 2 Related Work

The computation-allocation problem lies at the intersection of four established lines of work: numerical IK, learned multi-solution proposal, hybrid refinement, and per-instance solver selection. Their complementary strengths define both the building blocks of CG-HIK and the allocation question that remains.

### 2.1 Numerical, Analytical, and Constrained IK

Classical IK spans exact structural solutions and general iterative updates. Geometric and analytical methods exploit screw structure and subproblem decomposition, providing rapid branch enumeration for suitable manipulators [5–7]. Jacobian inverse and pseudoinverse methods generalize across chains, while DLS and singularity-robust inverses stabilize updates near rank loss [8–10]. Modern numerical extensions vary the step, run complementary solvers, batch initializations, or encode viability constraints [11–15]. RelaxedIK shows how pose, continuity, collision, and singularity objectives can be treated jointly as online motion synthesis [1], while recent analysis distinguishes local numerical conditioning from the topology of branch transitions [34]. These methods make robust IK a portfolio of complementary local and recovery mechanisms. CG-HIK adds a learned allocation policy over that fixed portfolio.

### 2.2 Learned Multi-Solution and History-Conditioned IK

Learned IK has progressed from point regression toward explicit representations of a set-valued inverse. Conditional normalizing flows, graph-equivariant generators, invertible modular networks, multiple-instance models, diffusion, and permutation-invariant set prediction improve diversity or structural transfer [17–22]. History-conditioned models add a reference configuration or predict a joint increment, making branch preference explicit rather than averaging globally separated solutions [3, 4, 23]. The diffusion and MimicIK records are treated here as preprints, and the sequential model as an online forthcoming record. Collectively, these methods show the value of learned proposal distributions. CG-HIK uses such a proposal upstream, but evaluates it through downstream numerical pathways and never treats a neural prediction as an accepted command.

### 2.3 Learned Proposals with Numerical Refinement

Hybrid pipelines combine the global coverage of learning with the precision and constraint handling of numerical geometry. IKFlow and CycleIK refine learned solutions numerically [17, 24]; CppFlow performs path-

level proposal, search, and refinement [2]; and diffusion priors can seed or guide trajectory optimization [25, 26]. Execution-time-aware redundancy selection, graph warm starts, and neuro-fuzzy approximators likewise make learned outputs useful through their effect on a downstream solver [27–29]. We retain this cooperative architecture but shift the learned target from the joint command to the amount and entry of numerical work. This enables an interpretable system comparison: the primary baselines and CG-HIK share proposals, solvers, fallback, budgets, and verification, differing in their computation-allocation rule.

## 2.4 Adaptive Computation and Solver Routing

Per-instance algorithm selection maps instance features to algorithms with different cost and success profiles [32]. Benchmark libraries formalize the corresponding instance–algorithm performance matrix and the distinction between a selected system and its empirical oracle [33]. Calibration and ensembles provide tools for controlling predictive confidence [35, 36], while runtime failure detection motivates conservative fallback under uncertainty [31]. Within IK, seed selection and reselection already demonstrate that basin choice and failure recovery affect runtime [30]. CG-HIK applies per-instance algorithm selection to the entries of a shared IK portfolio, using observed terminal success and query-conditioned P95 cost as routing targets. This places learning at the computation-allocation layer while retaining the proposal, numerical refinement, and verification mechanisms established by prior work. We formalize that decision next.

## 3 Problem Formulation

### 3.1 Online query and command contract

At control step  $t$ , an IK query is

$$\xi_t = \left( x_t^d, q_{t-1}, \Delta t \right), \quad (1)$$

where  $x_t^d = (p_t^d, R_t^d) \in SE(3)$  is the desired end-effector pose,  $q_{t-1} \in \mathcal{Q}$  is the previously accepted configuration, and  $\Delta t$  is the command interval. A numerical pathway proposes  $q_t$ . The shared deterministic contract is

$$\begin{aligned} \mathcal{V}(q_t; \xi_t) = & \mathbf{1} \left[ \|p(q_t) - p_t^d\|_2 \leq \epsilon_p \right] \\ & \wedge \mathbf{1} \left[ d_R(R(q_t), R_t^d) \leq \epsilon_R \right] \\ & \wedge \mathbf{1} \left[ q_{\min} - \epsilon_q \leq q_t \leq q_{\max} + \epsilon_q \right] \\ & \wedge \mathbf{1} \left[ |\Delta q_t| \leq \dot{q}_{\max} \Delta t + \epsilon_v \right] \\ & \wedge \mathbf{1} [q_t \text{ is finite}]. \end{aligned} \quad (2)$$

The orientation distance  $d_R$  is the norm of the rotation logarithm. This is a kinematic command contract, not a universal safety certificate: it contains no collision, torque, contact, or tracking model.

### 3.2 Entry actions and tail-cost objective

Let  $\mathcal{A} = \{\text{easy}, \text{medium}, \text{hard}\}$  denote entry actions into a nested numerical cascade. For query  $\xi$  and entry  $a$ , let  $Y(\xi, a) \in \{0, 1\}$  be terminal verified success and let  $T(\xi, a)$  be the observed action-pathway wall-clock latency, including proposal preparation, numerical solving, verification, and pathway framework overhead. The learned routing decision itself is excluded from these development labels because each action is forced during collection. Repeated measurements define conditional latency quantiles  $Q_\tau(T \mid \xi, a)$ . Fresh system measurements instead enclose the complete runtime, including the router’s fixed batch-one overhead.

The learned policy predicts  $\hat{p}_s(\xi)$  and entry-specific  $\hat{Q}_{0.50}(\xi, a)$  and  $\hat{Q}_{0.95}(\xi, a)$ . Given a calibrated success threshold  $\pi_{\min}$  and deadline  $D$ , the eligible set is

$$\mathcal{A}(\xi) = \left\{ a \in \mathcal{A} : \hat{p}_s(\xi) \geq \pi_{\min}, \hat{Q}_{0.95}(\xi, a) \leq D \right\}. \quad (3)$$

When this set is nonempty, the idealized routing rule is

$$a^*(\xi) = \arg \min_{a \in \mathcal{A}(\xi)} \hat{Q}_{0.95}(\xi, a), \quad (4)$$

with a conservative tie rule described below. Empty eligibility does not itself imply infeasibility: the policy must distinguish confident portfolio failure from uncertainty.

### 3.3 Research questions

The evaluation links three questions.

**RQ1—Allocation opportunity.** How large and stable are pathway-cost differences when fixed entries, the learned policy, and an empirical oracle are compared on the same queries?

**RQ2—Entry choice, rejection, and tail-aware selection.** On identical fresh queries, does learned entry choice outperform hard entry and a simple rule, and does P95-based selection improve on P50-based selection once actual runtime overhead is included?

**RQ3—Successful continuous computation.** Do savings persist while completing continuously feasible trajectories, how much comes from handling failures, and how does performance compare with TRAC-IK?

The method defines the allocation and acceptance decisions first. The experiments then separate these mechanisms with fixed-query controls and method-specific closed-loop state.

## 4 CG-HIK Method

CG-HIK separates the choice of computational effort from the generation and acceptance of a joint command. Figure 1 follows that separation from the shared numerical portfolio, through pathway observations and tail-cost routing, to deterministic verification.

### 4.1 Shared Numerical Solver Portfolio

An early-terminating cascade can still pay for an unnecessary prefix before reaching a sufficient stage. We therefore treat the three actions as alternative entries into the same escalating cascade (Figure 1). The easy entry begins with one DLS update initialized at  $q_{t-1}$ . If verification fails, it escalates to medium and then hard. Medium begins with the best learned history-conditioned candidate and a bounded DLS refinement, then escalates to hard. Hard runs longer DLS attempts from the learned candidate and  $q_{t-1}$  and, if needed, a bounded trust-region-reflective fallback from heterogeneous previous, learned, and retrieved seed-bank configurations. Duplicated seeds are removed. The Fixed robust cascade always enters at easy; the Fixed hard-entry cascade starts directly at hard.

For a DLS step with task residual  $e$  and Jacobian  $J$ , the update has the weighted, damped form

$$\Delta q = J_w^\top \left( J_w J_w^\top + \lambda^2 I \right)^{-1} e_w, \quad (5)$$

with adaptive damping, joint-step clipping, and joint-limit clipping. The shared portfolio uses the same tolerances, budgets, seed bank, fallback, and verifier for all methods (Table 1). Entry names describe execution order, not latent difficulty; an easy entry can still traverse the complete cascade after local failure.

### 4.2 History-Conditioned Candidate Proposal

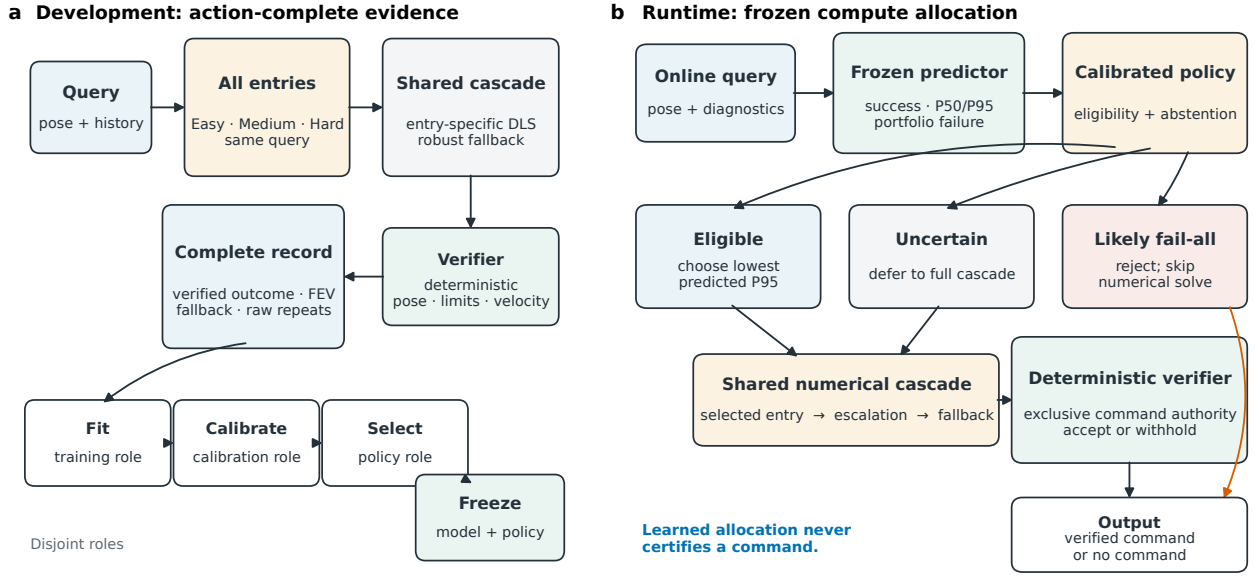
Previous-state continuation is inexpensive in a favorable basin but can be a poor initializer after a difficult target or branch transition. The hard pathways therefore need a stronger proposal without allowing that proposal to become a command source. A 5-member feed-forward ensemble proposes bounded joint increments conditioned on the previous configuration and desired pose. Each member contains 3 hidden layers of width 256 and predicts a normalized increment clipped to a maximum joint-space step. The ensemble is not used as an acceptance mechanism. Its best candidate is selected by a composite score over position and orientation residual, joint step, and joint-limit penalty, then supplied to medium/hard numerical refinement.

The frozen implementation prepares these learned candidates on every query, including easy-entry and rejected queries. Routing can skip numerical prefixes; it does not remove this proposal-generation cost. The direct hard baseline retains the same proposals but omits diagnostic features and predictions it does not need.

The routing feature vector contains 9 diagnostics available after candidate preparation: learned-candidate position and orientation residual, mean and maximum ensemble uncertainty, candidate Jacobian minimum singular value, candidate joint-limit margin, candidate-to-history joint step, and the position and orientation step from the current end-effector pose. These features describe both the proposal and the local query geometry. No feature relaxes the verifier.

### 4.3 Action-Complete Pathway Data Collection

Hand-assigned easy or hard labels cannot identify the least costly sufficient entry once escalation, verification, and fallback are included. For each development query, we therefore execute easy, medium, and hard entries rather than infer a label from a heuristic category. Every action retains repeated, Latin-rotated wall-clock



**Figure 1: CG-HIK framework.** During development (left), the same query is executed from every cascade entry and all verifier outcomes, FEV, fallback events, and raw timing repeats are retained. The compact predictor is fitted and calibrated on disjoint development roles. At runtime (right), the learned policy selects one eligible entry, rejects only high-confidence portfolio failure, or defers uncertainty to the complete cascade. Numerical solvers generate commands; the common deterministic verifier alone authorizes return.

measurements and the terminal verifier result, FEV, iterations, fallback status, executed stages, and command. A fixed-cascade control arm confirms that entering at easy reproduces the semantics of the primary baseline. The resulting matrix is action-complete: all alternative pathways were observed for the same query.

Because every entry is observed rather than imputed, we refer to this target as *action-complete pathway supervision*. Data roles are separated by query identity: model fitting, probability/OOD calibration, and policy selection use disjoint development subsets, and no fresh test outcome participates in training or threshold selection.

#### 4.4 Shared Success and Action-Specific Latency Prediction

All entries eventually share the same terminal fallback and verifier. The action-complete observations showed exact terminal semantic invariance across entries, so separate action-success predictions would introduce distinctions absent from the labels. This is an observed development regularity, not a guarantee for arbitrary queries or for later states reached by different closed-loop methods. Entry choice nevertheless changes which prefixes are executed and therefore changes cost, even when the final command is the same. Rare failed prefixes and fallback calls create the latency tail, which motivates predicting P95 as well as the median.

The predictor is a SiLU multilayer perceptron with 9 inputs and 2 hidden layers of width 32; its hidden representation is shared by all heads. It has (i) one calibrated terminal-success logit broadcast across the three entries, (ii) six positive outputs for entry-specific P50 and non-crossing P95 latency, and (iii) a calibrated fail-all logit. P50 and P95 are optimized with pinball loss against all raw timing repeats rather than unstable single-run winners. Platt scaling is fitted only on the calibration role [35]. Development latency labels exclude the router’s own inference cost because every action is forced during collection; the fresh end-to-end measurements include that cost and expose its fixed P50 overhead.

#### 4.5 Tail-Latency-Aware Routing Policy

Low predicted latency is useful only when the selected pathway is also likely to be sufficient. After calibration, an entry is therefore eligible when the shared success probability is at least the prespecified minimum and its predicted P95 does not exceed the 20 ms deadline. The policy selects the eligible entry with minimum predicted P95. If the predicted advantage over the most conservative eligible entry is smaller than the prespecified tie margin, it keeps the conservative entry. This prevents small, noisy predicted differences from inducing an aggressive skip.

The policy, thresholds, model weights, and numerical budgets remain fixed in all original and supplementary evaluations. The supplementary geometry and P50-selection controls change the specified decision rule only;

**Table 1: Shared solver portfolio and evaluation configuration.** All evaluated internal methods use these numerical and verification settings; only allocation changes. TRAC-IK retains the same public verifier but uses its own numerical implementation. Values are generated from the original configurations.

| Component         | Evaluation setting                     | Role                                     |
|-------------------|----------------------------------------|------------------------------------------|
| Easy entry        | Previous state                         | 1 DLS iteration                          |
| Medium entry      | Learned candidate                      | 1 DLS iteration                          |
| Hard entry        | 1 learned candidate + previous state   | 25 DLS iterations per seed               |
| Robust fallback   | Previous + learned + up to 2 retrieved | TRF, max 100 FEV per seed                |
| Verifier          | Shared for every method                | 0.001 m / 0.008727 rad                   |
| Routing predictor | 9 features; 32x32 MLP                  | shared success + action-specific P50/P95 |

they are not new trained methods.

#### 4.6 Deterministic Verification, Reject, and Conservative Defer

Predicted pathway cost or success is not proof that a joint command is valid. For every numerical path, reported convergence alone is insufficient: the returned configuration must pass Eq. (2), or the runtime escalates or reports failure. The learned system cannot override that decision. The fresh trajectory evaluator additionally reruns the same contract independently on every reported accepted command and counts any discrepancy as a contract violation.

Some queries also make every available pathway predictably futile. The predictor therefore estimates the probability that the complete portfolio fails. A query can be rejected only when no numerical entry is eligible, the fail-all probability exceeds the fixed threshold, and the embedding is not flagged as OOD. Reject executes zero numerical stages and returns no command. Uncertainty is treated differently from predicted futility: an OOD query, or one lacking sufficient fail-all agreement, is deferred to the complete Fixed robust cascade. This defer outcome is an auxiliary conservative mechanism rather than the main route by which CG-HIK saves computation.

#### 4.7 Batch-One Cost Accounting

Per-query routing is worthwhile only if its implementation cost does not erase the numerical work it avoids. To preserve batch-one cost accounting, the predictor and policy are exported as one runtime module. The fresh evaluations include this routing cost, so measured savings reflect the deployed execution path rather than solver work alone.

These choices create three empirical tests rather than guarantees: whether pathway differences are stable enough to exploit, whether they repay prediction overhead, and whether altered commands preserve future recovery along a trajectory.

### 5 Experimental Protocol

The original tests establish performance relative to the initial fixed cascade. The supplementary evaluation probes the mechanism with stronger fixed-entry controls, explicit feasibility witnesses, and an external solver. No supplementary outcome is used to retrain the model or select its thresholds, budgets, or fallback.

#### 5.1 Shared implementation and timing

Panda and UR5e use the repository’s URDFKinematics forward-kinematics and geometric Jacobian implementation, without a physics simulator. SciPy supplies the bounded trust-region-reflective fallback [37]; PyTorch supplies the learned modules [38], and scikit-learn utilities support the original calibration stack [39]. The supplementary measurements use an Intel Core i7-13700K, an NVIDIA Quadro RTX 8000 for candidate generation, PyTorch 2.11.0+cu128, eight intra-op threads and one inter-op thread. Methods are measured sequentially, with no concurrent benchmark.

Whole-call timing includes input conversion, required candidate preparation, diagnostics, prediction, decision, numerical solving, fallback, and in-runtime verification. GPU synchronization encloses applicable calls. Initialization, warmup, serialization, and independent verification replay are outside the timed interval. The supplementary always-hard wrapper omits unused routing features and prediction while retaining the frozen hard-stage commands and budgets; smoke comparisons confirmed command, FEV, stage, and fallback

**Table 2: Controlled allocation rules.** All internal numerical commands use the frozen solver portfolio and public verifier. Original eligibility means the same success threshold and predicted-P95 deadline criterion. Defer always enters the complete easy-entry cascade.

| Method             | Reject/defer                         | Eligibility                | Non-abstained entry                                         |
|--------------------|--------------------------------------|----------------------------|-------------------------------------------------------------|
| Always-hard        | Neither                              | None                       | Hard; no routing diagnostics or prediction                  |
| Geometry rule      | Original                             | Determines abstention only | Easy if both step thresholds pass, otherwise hard           |
| Reject-only + hard | Original                             | Original                   | Hard                                                        |
| Routing-only       | Reject becomes hard; defer unchanged | Original                   | Original P95 ordering                                       |
| P50-selection      | Original                             | Original                   | P50 ordering with the same conservative-entry tie framework |
| CG-HIK             | Original                             | Original                   | Frozen P95 ordering and tie rule                            |

agreement with the original wrapper. Historical pathway-label timings and the original profiling wrapper are therefore not substituted for this directly measured deployment baseline.

## 5.2 Pathway observations and original result decomposition

The original action-complete corpus contains 40000 queries, each with 5 interleaved calls per entry. Fitting, calibration, and policy-selection roles are disjoint. To compare fixed easy, medium, hard, the frozen policy, and an empirical oracle on one population, we use only successful, non-abstained policy-validation queries: 2,031 for Panda and 2,049 for UR5e. We report numerical-plus-verification cost separately from the historical label pipeline, which also contains candidate preparation and a constant-risk profiling shell. Deployment routing overhead was not measured on that population and is left unreported. An oracle chosen from five-repeat empirical P95 is a noisy in-sample diagnostic, not a known true optimum.

We also partition every original fresh trajectory by the two observed completion outcomes: both complete, only CG-HIK completes, only always-hard completes, and neither completes. All 150 frames remain in each trajectory’s latency and FEV sum. Group counts, costs, first failures, and reject/defer/fallback events add back to the original totals. This is explicitly post-hoc descriptive decomposition; outcome-conditioned groups are not causal treatment strata.

## 5.3 Identical-query mechanism controls

Each robot receives 3,000 newly generated point queries: 1,000 local continuations, 500 near-singular, 500 near-limit, 500 difficult feasible, and 500 constructively inexecutable requests. The first 2,500 retain a reference joint configuration that passes the unchanged verifier. Near-singular configurations are selected by geometric Jacobian conditioning; feasible targets are generated by FK from bounded joint increments. Inexecutable target positions lie outside a global reach bound obtained from the sum of URDF segment-translation lengths for these fixed/revolute chains. No query is labeled infeasible because an evaluated method fails.

All methods receive identical target pose, previous configuration, and interval. The previous state is not updated between point calls. Each query has three randomized, cyclically interleaved repetitions of the six internal controls in Table 2, plus all four TRAC-IK budgets. Geometry thresholds are selected using old calibration labels only: 0.02 m/0.01 rad for Panda and 0.005 m/0.025 rad for UR5e. The geometry control shares the frozen reject/defer predictor; it tests the need for learned *entry ordering*, not the elimination of all model cost.

A prespecified 500-query feasible subset per robot contains 200 local and 100 from each other feasible family. Each forced entry is measured ten times. Five even interleaved repeats select the lowest empirical P95 among entries successful on all five; the five odd repeats evaluate that choice, and then the roles swap. Queries without a successful selection entry are reported separately. We measure preference stability, held-out cost relative to hard, and the fraction whose entry means differ by at most 0.15 ms. The last threshold is a descriptive near-equivalence diagnostic, not a trained parameter. No oracle choice feeds back into the method.

## 5.4 External solver under the same command contract

The external comparison uses official `trac_ik_lib` 2.2.0 [11] in Speed mode.<sup>1</sup> Its KDL and nonlinear optimization workers run concurrently. The unmodified official sources are compiled with GNU C++ 9.5.0 and NLOpt 2.7.1 behind a small C interface. The adapter uses the same URDF, base/tool chain, joint ordering, and previous-state initialization. KDL FK agrees with the public model below  $7 \times 10^{-16}$  on 100 configurations per robot.

For every query, `setKDLlimits` intersects URDF bounds with the allowed one-frame joint interval. Both supplied chains have bounded revolute active joints, including UR5e’s wide but finite bounds; they are not wrapped as continuous joints. The adapter separately supports genuine continuous variables using a local unwrapped search interval. Internal epsilon is  $10^{-5}$  per Cartesian component with zero additional Twist tolerance, stricter than the public pose tolerances. A nonnegative solver return still requires the public verifier. Per-query bound updates and their internal solver reconstruction are included in timing; model loading is not. TRAC-IK FEV is unavailable under the internal definition and is not compared.

All point results retain the fixed 5/20/100/400 ms search budgets. Curves count a success only when a verified command actually returns within the displayed elapsed time. The trajectory budget is selected independently on 120 old calibration queries per robot, maximizing success, then minimizing measured total time, then budget; both robots select 5 ms before point-test outcomes are opened.

## 5.5 Continuously feasible reference trajectories

Each robot has 40 new trajectories, ten each from smooth motion, a low-manipulability neighborhood, joint-limit approach and return, and high-curvature/faster motion. Every trajectory has 150 target frames at  $\Delta t = 0.02$  s. A reference joint path is generated first and converted to targets through FK. Geometric amplitude scaling enforces joint and one-step velocity constraints before evaluation; every reference transition passes the public verifier. Seeds and trajectory identities are new and fixed before comparison, with query-hash isolation recorded in the manifest. No method’s success rate is used to retain or reject a path.

Always-hard, routing-only, CG-HIK, and calibration-selected TRAC-IK start at the same reference initial configuration. Online calls receive no future reference joints, extra reference seed, or branch label. Each method then evolves its own state: acceptance updates the state, failure holds it, and targets advance through all 150 fixed indices. Reference feasibility proves the existence of a path, not recoverability from every state an online method might reach.

Geometric completion requires acceptance at every frame. We separately report the fraction of calls accepted within their measured 20 ms interval and the number of trajectories accepted within 20 ms on every frame. A geometrically accepted late command still updates state in this kinematic sequence evaluation, but does not count as timely acceptance. Thus nominal  $\Delta t$  does not establish real-time control at 50 Hz. Lost and gained completion UIDs are reported, not suppressed by requiring identical completion vectors.

## 5.6 Statistical units and preservation

Point uncertainty uses paired queries, retaining all three technical repetitions within each resampled query. Trajectory uncertainty uses complete paired trajectories, never 150 independent frames. We report ratios of total cost and success differences with 95% percentile intervals from 4,000 paired, family-stratified bootstrap resamples. Frame quantiles are empirical whole-call quantiles; paired quantile intervals resample query clusters. These are descriptive intervals, without multiplicity-adjusted significance claims or a composite gate.

The original results and source data are unchanged. A first incomplete Panda supplementary point run was interrupted after code inspection found a redundant P95 sorting operation in the control adapter. That partial run was excluded in full and preserved separately; no aggregate outcomes were inspected or used. The corrected controls perform one inference and only the ranking required by their definition. Fresh identities, calibration choices, the main method, and numerical settings remained fixed. The complete supplementary measurements and findings report precede this manuscript revision.

# 6 Results

The evidence proceeds from the allocation opportunity to direct mechanism controls and then to continuous execution. Original held-out results remain unchanged (Appendix A); the new analyses test what those aggregate gains mean.

<sup>1</sup>Upstream source commit: 90162ac2ecc6ea8f88c6e99df6ee01efd217a3fb.

**Table 3: Same-population pathway comparison.** Times are milliseconds. *Numerical* includes verification; *label pipeline* additionally includes candidate preparation and the historical constant-risk profiling shell. The last measured column averages noisy per-query P95 estimates from five calls. Actual deployed routing overhead was not measured here (–) and is not imputed. The empirical oracle minimizes the observed label-pathway P95, not the separate numerical-cost component.

| Robot | Choice           | Queries | Mean numerical | Mean label pipeline | Mean query P95 | Deployment |
|-------|------------------|---------|----------------|---------------------|----------------|------------|
| Panda | Fixed easy       | 2031    | 1.060          | 2.237               | 2.437          | –          |
| Panda | Fixed medium     | 2031    | 0.932          | 2.111               | 2.312          | –          |
| Panda | Fixed hard       | 2031    | 0.811          | 1.993               | 2.197          | –          |
| Panda | CG-HIK           | 2031    | 0.811          | 1.990               | 2.190          | –          |
| Panda | Empirical oracle | 2031    | 0.810          | 1.947               | 2.055          | –          |
| UR5e  | Fixed easy       | 2049    | 1.001          | 2.130               | 2.362          | –          |
| UR5e  | Fixed medium     | 2049    | 0.810          | 1.939               | 2.158          | –          |
| UR5e  | Fixed hard       | 2049    | 0.630          | 1.765               | 1.987          | –          |
| UR5e  | CG-HIK           | 2049    | 0.630          | 1.764               | 1.984          | –          |
| UR5e  | Empirical oracle | 2049    | 0.637          | 1.726               | 1.852          | –          |

**Table 4: Independent-fold empirical entry diagnostic.** Each robot has 500 prespecified feasible queries and ten calls per entry. Even/odd groups select and evaluate separately, then swap. Stability uses queries with a successful choice in both folds; the near-equal mean fraction uses all 500. The held-out ratio includes candidate and forced-path costs, without learned routing overhead.

| Robot | Two valid folds | Stable (%) | Near-equal means (%) | Held-out time / hard [95% CI] |
|-------|-----------------|------------|----------------------|-------------------------------|
| Panda | 471             | 41.61      | 57.60                | 0.9978 [0.9949, 1.0002]       |
| UR5e  | 500             | 43.80      | 78.00                | 1.0015 [0.9988, 1.0042]       |

## 6.1 RQ1: pathway differences and the stability of empirical winners

RQ1 asks whether pathway costs provide a stable opportunity for learned allocation, rather than merely different winners under noisy timing. Table 3 compares all five choices on the same policy-validation queries. Starting at hard already removes much of the numerical prefix cost paid by easy. The learned policy is close to hard in numerical-plus-verification cost: 0.811 versus 0.811 ms on Panda, and 0.630 versus 0.630 ms on UR5e at the displayed precision. Historical pipeline means differ slightly, but do not include the deployed learned decision. These measurements therefore do not establish an end-to-end advantage for learned routing.

The ten-repeat diagnostic tests whether an empirical preference survives new measurements of the same input. Among queries with a successful entry in both selection folds, preference agreement is 41.61% for Panda and 43.80% for UR5e. Panda has 29 additional queries with no successful entry in either fold; they are not counted as stable successful preferences. The cross-fitted mean-time ratios relative to hard are 0.9978 and 1.0015, with both intervals spanning 1 (Table 4). Ten-repeat entry means are within 0.15 ms for 57.6% and 78.0% of all diagnostic queries, respectively. The corresponding fractions for entry P95 are only 9.2% and 23.6%, illustrating how much less stable an upper quantile can be with few repeats.

Thus pathway prefixes have measurable costs, but low regret in this portfolio is also compatible with nearly equivalent entries and unstable empirical winners. RQ1 does not by itself establish a useful learned advantage beyond hard entry; RQ2 measures that advantage directly, including the cost of making the decision.

## 6.2 RQ2: learned entry choice, rejection, and P95 selection

RQ2 asks whether learned entry ordering improves actual computation over a strong fixed entry or a simple rule, and whether its P95 objective matters. The original point test preserved verified success while reducing FEV and P95 relative to the fixed easy-entry cascade (Appendix A.2). The supplementary comparison is more demanding: it uses a direct hard deployment baseline, exact matched inputs, and feasibility witnesses independently of all evaluated methods.

All six internal controls achieve the same feasible-query verified success: 94.92% for Panda and 100% for UR5e (Table 5). All witness-feasible queries have zero learned false rejection. The 127 Panda queries that fail under the internal controls remain labeled feasible: a solver failure does not invalidate their witnesses. Removing early rejection makes no difference to feasible-point commands because no rejection occurs in this subset.

Neither full nor routing-only CG-HIK reduces feasible-point time relative to hard. Full mean time increases by 3.33% on Panda and 29.32% on UR5e; routing-only increases it by 3.34% and 29.35%. The paired full-minus-hard mean differences are 0.503 ms [0.486, 0.525] and 0.448 ms [0.440, 0.455]. FEV is nearly unchanged rather

**Table 5: Six internal strategies on witness-feasible points.** There are 2,500 paired queries per robot, with three interleaved calls each. Latency is in ms; success and timely acceptance use the same verifier. No witness-feasible query is rejected by the learned policy.

| Robot | Method             | Verified (%) | Mean FEV | P50   | P95    | P99     | Within 20 ms (%) |
|-------|--------------------|--------------|----------|-------|--------|---------|------------------|
| Panda | Always-hard        | 94.92        | 20.559   | 1.501 | 89.893 | 347.605 | 93.32            |
| Panda | Geometry rule      | 94.92        | 20.572   | 1.984 | 90.321 | 348.077 | 93.31            |
| Panda | Reject-only + hard | 94.92        | 20.559   | 1.982 | 90.633 | 348.045 | 93.29            |
| Panda | Routing-only       | 94.92        | 20.583   | 1.986 | 90.548 | 348.007 | 93.31            |
| Panda | P50-selection      | 94.92        | 20.702   | 1.989 | 90.765 | 348.019 | 93.31            |
| Panda | CG-HIK             | 94.92        | 20.583   | 1.984 | 90.488 | 348.073 | 93.31            |
| UR5e  | Always-hard        | 100.00       | 3.462    | 1.342 | 2.021  | 3.668   | 100.00           |
| UR5e  | Geometry rule      | 100.00       | 3.468    | 1.788 | 2.486  | 4.474   | 100.00           |
| UR5e  | Reject-only + hard | 100.00       | 3.462    | 1.789 | 2.453  | 4.016   | 100.00           |
| UR5e  | Routing-only       | 100.00       | 3.464    | 1.794 | 2.464  | 4.236   | 100.00           |
| UR5e  | P50-selection      | 100.00       | 3.529    | 1.792 | 2.508  | 4.292   | 100.00           |
| UR5e  | CG-HIK             | 100.00       | 3.464    | 1.793 | 2.479  | 4.020   | 100.00           |

**Table 6: Paired feasible-point effects.** Ratios below 1 favor the numerator. Intervals resample paired queries while retaining their three calls.

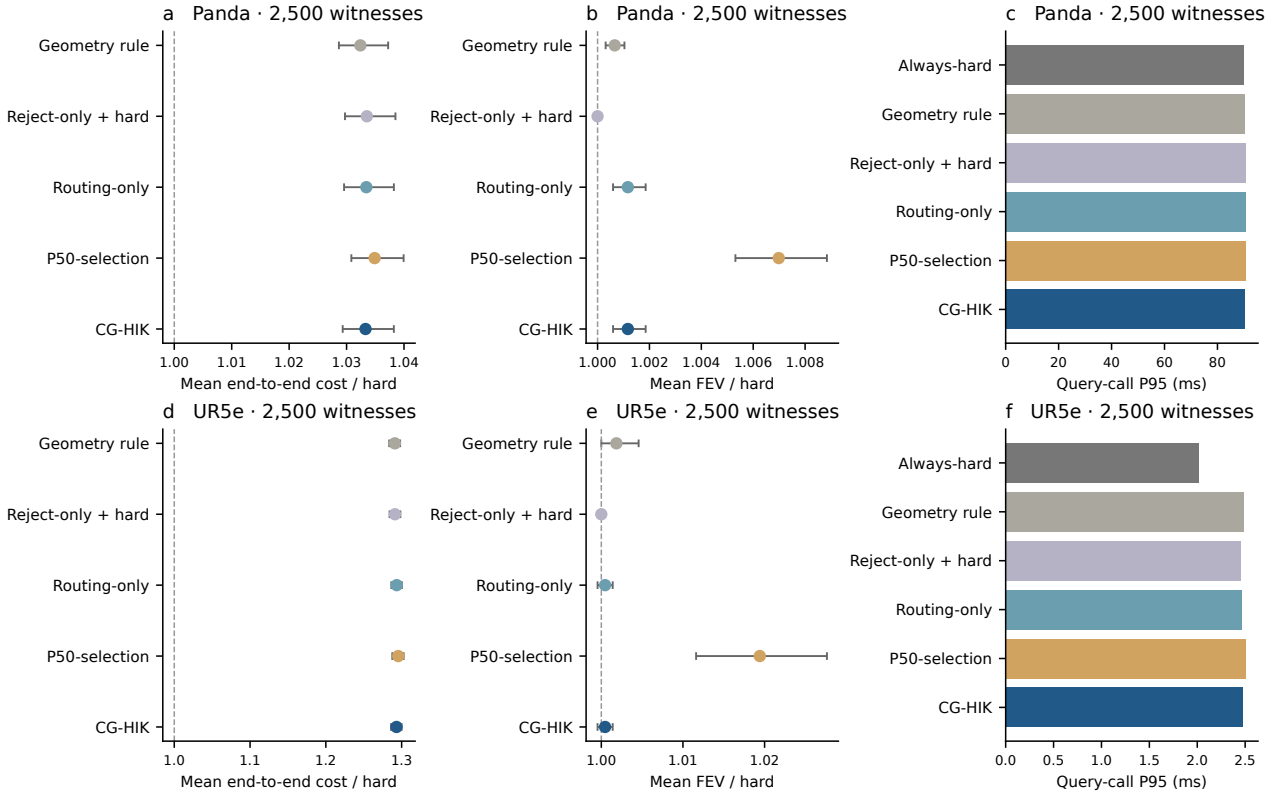
| Robot | Comparison                 | Mean time ratio [95% CI] | Mean FEV ratio [95% CI] |
|-------|----------------------------|--------------------------|-------------------------|
| Panda | Routing-only / Always-hard | 1.0334 [1.0296, 1.0382]  | 1.0012 [1.0006, 1.0019] |
| Panda | CG-HIK/ Always-hard        | 1.0333 [1.0293, 1.0382]  | 1.0012 [1.0006, 1.0019] |
| Panda | CG-HIK/ Reject-only + hard | 0.9998 [0.9983, 1.0013]  | 1.0012 [1.0006, 1.0019] |
| Panda | CG-HIK/ P50-selection      | 0.9985 [0.9970, 0.9999]  | 0.9942 [0.9925, 0.9958] |
| Panda | CG-HIK/ Geometry rule      | 1.0009 [0.9997, 1.0022]  | 1.0005 [1.0001, 1.0011] |
| UR5e  | Routing-only / Always-hard | 1.2935 [1.2864, 1.3003]  | 1.0005 [0.9995, 1.0014] |
| UR5e  | CG-HIK/ Always-hard        | 1.2932 [1.2861, 1.3000]  | 1.0005 [0.9995, 1.0014] |
| UR5e  | CG-HIK/ Reject-only + hard | 1.0016 [0.9979, 1.0053]  | 1.0005 [0.9995, 1.0014] |
| UR5e  | CG-HIK/ P50-selection      | 0.9983 [0.9942, 1.0022]  | 0.9814 [0.9737, 0.9890] |
| UR5e  | CG-HIK/ Geometry rule      | 1.0017 [0.9978, 1.0055]  | 0.9986 [0.9961, 1.0002] |

than reduced (Figure 2; Table 6). Full and the geometry or reject-only controls have very similar cost, with the mean-time ratio intervals spanning 1. Together with nearly unchanged numerical work, this is consistent with decision overhead outweighing saved prefixes: the simple rule still pays for the shared abstention predictor, whereas direct hard does not pay for unused routing diagnostics.

**Does P95 selection add value?** P95-based selection reduces FEV relative to P50-based selection by 0.58% on Panda and 1.86% on UR5e, with unchanged success. Panda also shows a small mean-time reduction of about 0.024 ms; the UR5e mean-time interval spans zero difference. These are measurable numerical-work effects, not evidence that the method needs a mean-latency-trained alternative. The control uses the existing median prediction. The P95 and P99 ratio intervals both span 1 on both robots (Table 7), so this experiment does not resolve a distinct tail-latency advantage of P95 selection.

**Separating rejection from entry choice.** On constructively inexecutable points, full CG-HIK rejects 44.0% of Panda queries and 12.8% of UR5e queries; the remaining requests defer to the full cascade. All methods return no accepted commands. Relative to hard, full CG-HIK saves 44.4% and 9.8% of total time on this subset. Geometry, P50-selection, and reject-only controls execute the same rejection/defer pattern and nearly the same cost, whereas routing-only incurs the hard work avoided by rejection. Thus aggregate savings can arise without a learned entry advantage on feasible queries. These failed-request costs are reported separately rather than pooled into the main feasible-point comparison.

**External success–time position.** TRAC-IK occupies a substantially faster feasible-point operating region (Figure 3; Table 8). Its four budgets give Panda verified success of 97.83–97.93%, above CG-HIK’s 94.92%, and UR5e success of 99.947–99.960%, slightly below CG-HIK’s 100%. Relative to the 5 ms configuration, full



**Figure 2: Entry mechanisms on the same feasible queries.** Left: mean complete-call cost relative to direct hard; middle: mean internal FEV relative to hard. Error bars are paired, query-cluster 95% bootstrap intervals. Right: empirical complete-call P95. The reference line at 1 identifies equal cost; different horizontal ranges reflect the robot-specific effect sizes. No constructed inexecutable points contribute to these panels.

**Table 7: Direct P95-versus-P50 decision ablation.** Full CG-HIK divided by P50-selection, with identical success thresholds, P95 eligibility, abstention, and tie-rule framework. Query-cluster intervals retain all within-query repetitions.

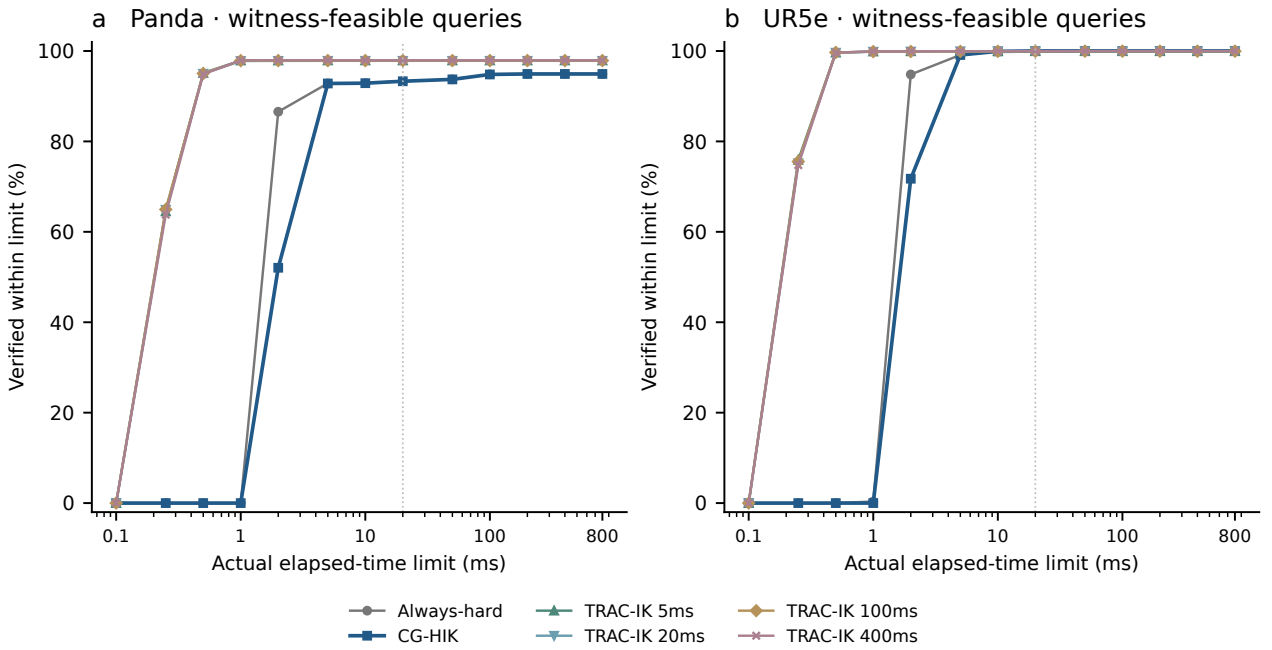
| Robot | Observed P95 ratio [95% CI] | Observed P99 ratio [95% CI] |
|-------|-----------------------------|-----------------------------|
| Panda | 0.9969 [0.9758, 1.0101]     | 1.0002 [0.9984, 1.0020]     |
| UR5e  | 0.9885 [0.9615, 1.0099]     | 0.9365 [0.8644, 1.0527]     |

CG-HIK’s mean-time ratios are 59.31 [52.20, 66.38] and 8.50 [8.37, 8.63]. The corresponding success differences are  $-2.987$  percentage points  $[-3.747, -2.253]$  for Panda and  $+0.053$  [0.000, 0.160] for UR5e. Longer search budgets change little on these feasible point inputs; they remain reported rather than selecting a test winner.

RQ2 therefore separates the mechanisms: learned entry selection does not beat hard or the geometry rule on these feasible points, P95 selection saves a small amount of FEV without a resolved tail advantage, and rejection reduces failed-request work. The next test asks whether those savings survive when each accepted command changes the following query’s state.

**Table 8: Official TRAC-IK on the same feasible points.** Budget is search time in ms, not a bound on complete-call time. Mean and P50/P95/P99 use actual elapsed calls. External FEV is not equated with internal FEV.

| Robot | Budget | Verified (%) | Mean ms | P50   | P95   | P99   | Within 20 ms (%) |
|-------|--------|--------------|---------|-------|-------|-------|------------------|
| Panda | 5      | 97.907       | 0.263   | 0.230 | 0.475 | 0.646 | 97.907           |
| Panda | 20     | 97.827       | 0.264   | 0.230 | 0.472 | 0.645 | 97.827           |
| Panda | 100    | 97.880       | 0.263   | 0.229 | 0.471 | 0.656 | 97.880           |
| Panda | 400    | 97.933       | 0.264   | 0.230 | 0.472 | 0.646 | 97.933           |
| UR5e  | 5      | 99.947       | 0.232   | 0.214 | 0.347 | 0.431 | 99.947           |
| UR5e  | 20     | 99.947       | 0.238   | 0.214 | 0.347 | 0.425 | 99.947           |
| UR5e  | 100    | 99.960       | 0.239   | 0.214 | 0.348 | 0.423 | 99.947           |
| UR5e  | 400    | 99.960       | 0.234   | 0.214 | 0.352 | 0.433 | 99.960           |



**Figure 3: Verified commands returned within actual elapsed time.** All 2,500 witness-feasible queries and three calls per method contribute. The four external curves largely overlap; all four are retained. Late successes only enter at their actual elapsed-time threshold. The dotted 20 ms line is a reporting threshold, not a hard upper bound supplied by CG-HIK.

### 6.3 RQ3: feasible continuous paths and the source of trajectory savings

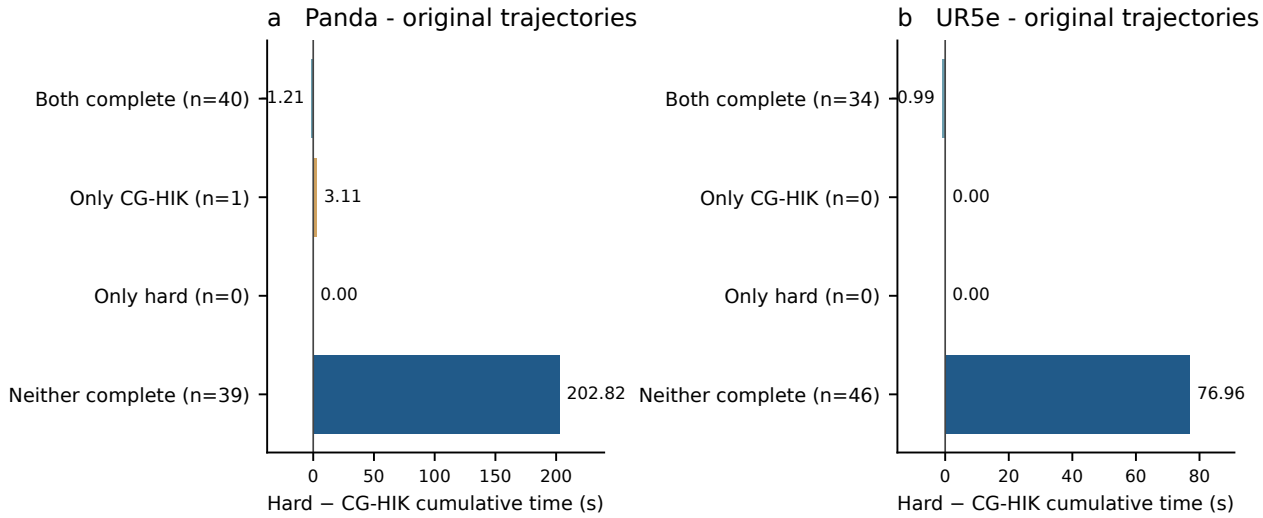
RQ3 asks whether lower cumulative computation corresponds to faster successful continuous solving. The original transition-rich evaluation reported lower aggregate time and FEV, with Panda completion 41/80 versus 40/80 for hard and UR5e 34/80 for both. Splitting those same frozen records by completion outcome changes the interpretation of that total (Table 9; Figure 4). Jointly completed paths are slower under CG-HIK: 11.88% on Panda and 12.45% on UR5e. The jointly unsuccessful groups account for 99.07% and 101.31% of net time savings, offsetting additional cost elsewhere. Panda’s single gained completion contributes a further 1.52%. These are exact descriptive allocations of observed cost, not estimates of a causal effect within outcome-selected groups.

**Reference feasibility and completion.** The supplementary reference paths remove uncertainty about the existence of a continuous feasible solution. They do not eliminate method-specific branch evolution. CG-HIK and routing-only each complete 34/40 Panda paths, versus 37/40 for hard; all three complete the same 36/40 UR5e paths (Table 10). Panda loses three hard-successful paths and gains none. Its completion-rate difference is  $-7.5$  percentage points  $[-15.0, 0.0]$ . No accepted command violates the public contract.

Full CG-HIK has lower observed Panda cumulative time, 55.346 to 39.003 s, but this accompanies the three lost completions. Its total-time ratio is 0.7047  $[0.3147, 2.4703]$ , and its FEV ratio is 0.7309  $[0.3410, 1.9626]$ . UR5e preserves completion but takes longer overall, 15.950 to 17.021 s: time ratio 1.0671  $[0.9262, 1.2859]$ , FEV

**Table 9: Post-hoc decomposition of the original frozen trajectories.** All fixed-length frames remain included. Four groups reconstruct the total time and FEV exactly. A contribution above 100% offsets extra cost in other groups; negative contributions increase total computation.

| Robot | Completion group | Paths | Hard time (s) | CG-HIK time (s) | Hard / CG-HIK FEV | Net savings (%) |
|-------|------------------|-------|---------------|-----------------|-------------------|-----------------|
| Panda | Both complete    | 40    | 10.142        | 11.347          | 16391 / 16082     | -0.59           |
| Panda | Only CG-HIK      | 1     | 3.404         | 0.290           | 7597 / 438        | 1.52            |
| Panda | Only hard        | 0     | 0.000         | 0.000           | 0 / 0             | 0.00            |
| Panda | Neither complete | 39    | 458.498       | 255.676         | 706261 / 383919   | 99.07           |
| UR5e  | Both complete    | 34    | 7.991         | 8.986           | 15367 / 15253     | -1.31           |
| UR5e  | Only CG-HIK      | 0     | 0.000         | 0.000           | 0 / 0             | 0.00            |
| UR5e  | Only hard        | 0     | 0.000         | 0.000           | 0 / 0             | 0.00            |
| UR5e  | Neither complete | 46    | 130.237       | 53.280          | 407353 / 132182   | 101.31          |



**Figure 4: Where the original cumulative-time savings occur.** Bars show hard minus CG-HIK time, summed over all frames in each trajectory group. Negative bars are retained. Most savings lie in paths neither method completes, whereas jointly completed paths pay additional time. This outcome-conditioned figure is descriptive, not causal.

ratio 0.9062 [0.7859, 1.0021]. The wide trajectory intervals reflect a few expensive paths rather than thousands of independent frame observations. P50 and P95 increase on both robots; P99 decreases in the full method. Routing-only takes more total time than hard on both robots (Table 11).

**Successful-path work and family structure.** Among jointly completed reference paths, full CG-HIK’s cumulative time increases by 18.23% on Panda (34 paths) and 32.12% on UR5e (36 paths). Panda FEV decreases from 16,607 to 15,108 on this group, but its fixed runtime overhead offsets that numerical-work reduction; UR5e FEV changes from 16,042 to 16,052. In the Panda jointly failed group, time savings of 27.641 s offset the extra 1.500 s on jointly completed paths and 9.799 s on the three lost completions. On UR5e, failed-group savings of 1.303 s do not offset 2.374 s extra on the jointly completed group.

The family results identify a positive local exception rather than a uniform effect (Table 12). Panda joint-limit-return retains 10/10 completions while reducing time by 5.58% and FEV by 25.53%; routing-only also saves time there. In contrast, Panda near-singular completion falls from 8/10 to 5/10 and cumulative time increases. UR5e near-singular completion is unchanged but computation increases. The smooth family’s large Panda aggregate saving includes a failed path and should not be interpreted as uniformly faster completed smooth motion.

**Branch evolution and the role of rejection.** All three lost Panda paths first return different, valid commands at frame 0: easy-entry CG-HIK and hard differ by approximately 0.00165–0.00187 rad in the most changed joint. At the first failures, frames 96, 120, and 125 (zero-indexed), their previous states differ from hard by approximately 0.078–0.087 rad. Each failure follows the complete easy–medium–hard cascade, not an early reject. Removing rejection loses the same three paths. The records therefore identify closed-loop sensitivity to accepted branch evolution, rather than rejection of an otherwise rescued command, as the relevant mechanism in these examples.

**Table 10: New continuously feasible reference paths.** Each robot has 40 trajectories of 150 frames. Timely completion requires every frame to return an accepted command within 20 ms; geometric completion has no such timing requirement. Cumulative time includes all calls, including failures and late returns. Frame latencies are milliseconds.

| Robot | Method         | Complete | Timely | Total (s) | Mean FEV | P50   | P95    | P99     |
|-------|----------------|----------|--------|-----------|----------|-------|--------|---------|
| Panda | Always-hard    | 37/40    | 36/40  | 55.346    | 16.034   | 1.413 | 2.071  | 253.679 |
| Panda | Routing-only   | 34/40    | 34/40  | 72.091    | 19.659   | 1.879 | 72.835 | 284.994 |
| Panda | CG-HIK         | 34/40    | 34/40  | 39.003    | 11.719   | 1.877 | 2.377  | 177.755 |
| Panda | TRAC-IK (5 ms) | 36/40    | 36/40  | 3.040     | –        | 0.234 | 5.067  | 5.168   |
| UR5e  | Always-hard    | 36/40    | 36/40  | 15.950    | 6.323    | 1.336 | 2.038  | 36.702  |
| UR5e  | Routing-only   | 36/40    | 36/40  | 18.669    | 6.402    | 1.779 | 2.456  | 37.316  |
| UR5e  | CG-HIK         | 36/40    | 36/40  | 17.021    | 5.730    | 1.777 | 2.277  | 34.678  |
| UR5e  | TRAC-IK (5 ms) | 39/40    | 39/40  | 1.699     | –        | 0.219 | 0.345  | 5.067   |

**Table 11: Paired reference-trajectory effects.** All ratios retain the same 40 full trajectory units per robot. Completion differences are percentage points, not frame-level differences.

| Robot | Comparison                 | Total time ratio [95% CI] | FEV ratio [95% CI]      | Completion difference [95% CI] |
|-------|----------------------------|---------------------------|-------------------------|--------------------------------|
| Panda | CG-HIK/ Always-hard        | 0.7047 [0.3147, 2.4703]   | 0.7309 [0.3410, 1.9626] | -7.5 [-15.0, 0.0]              |
| Panda | Routing-only / Always-hard | 1.3026 [1.0161, 3.3297]   | 1.2261 [0.9743, 2.6078] | -7.5 [-15.0, 0.0]              |
| Panda | CG-HIK/ Routing-only       | 0.5410 [0.2842, 0.9992]   | 0.5961 [0.3128, 1.0000] | 0.0 [0.0, 0.0]                 |
| UR5e  | CG-HIK/ Always-hard        | 1.0671 [0.9262, 1.2859]   | 0.9062 [0.7859, 1.0021] | 0.0 [0.0, 0.0]                 |
| UR5e  | Routing-only / Always-hard | 1.1705 [1.1157, 1.2882]   | 1.0125 [1.0005, 1.0236] | 0.0 [0.0, 0.0]                 |
| UR5e  | CG-HIK/ Routing-only       | 0.9117 [0.8073, 0.9991]   | 0.8950 [0.7693, 1.0000] | 0.0 [0.0, 0.0]                 |

Across all reference paths, full CG-HIK saves 45.90% of Panda and 8.83% of UR5e time relative to routing-only, with identical completion sets. At every full-method reject event (133 Panda and 68 UR5e frames), routing-only receives the identical complete query but its hard call does not return a verified command. This paired observation establishes avoided unsuccessful work for those calls, not a proof that no feasible command exists from that state.

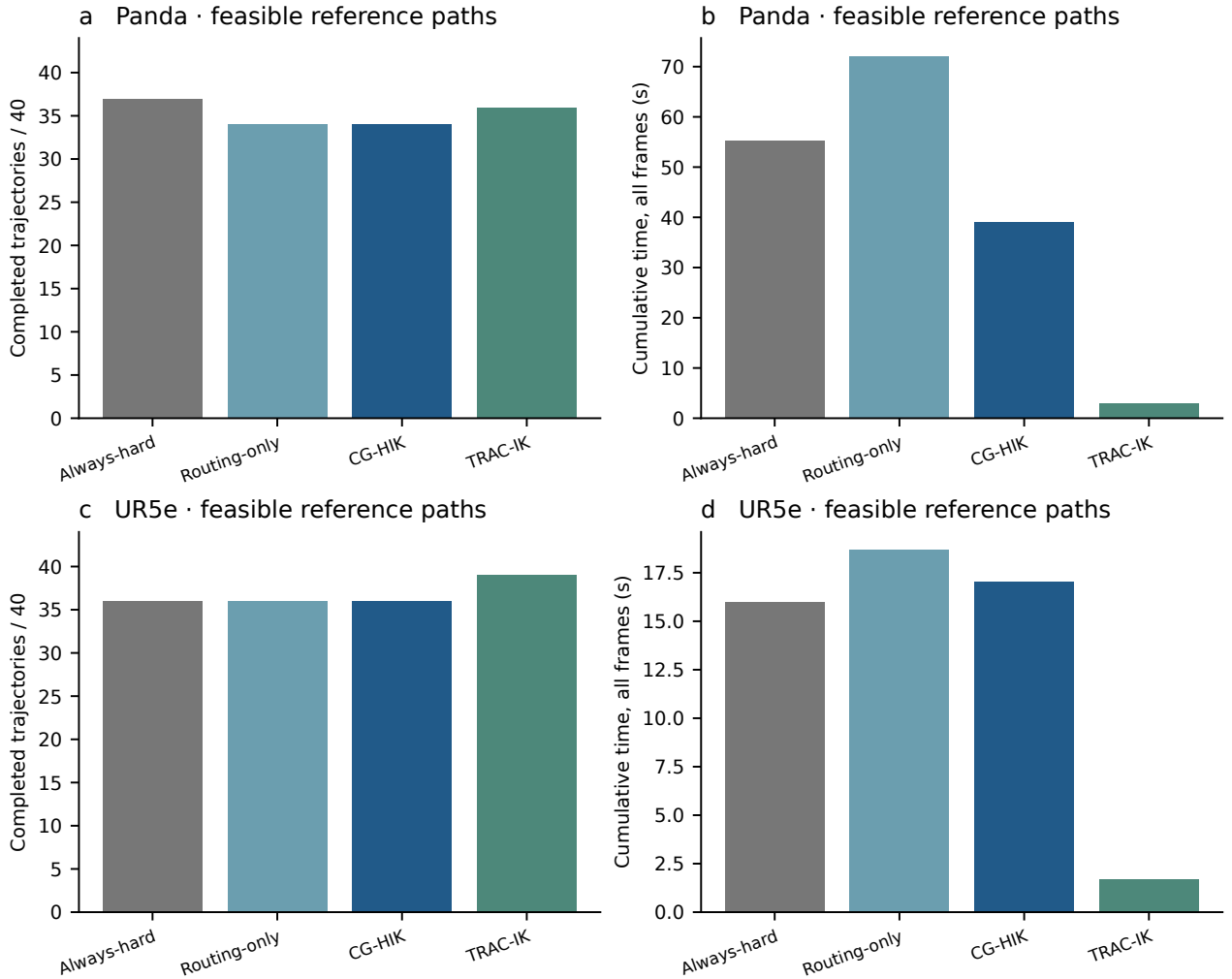
**External continuous performance and deadlines.** Calibration-selected 5 ms TRAC-IK completes 36/40 Panda and 39/40 UR5e reference paths, using 3.040 and 1.699 s in total (Figure 5). Full CG-HIK’s total-time ratios relative to it are 12.83 [6.44, 21.91] and 10.02 [7.08, 14.87]. Completion sets are not interchangeable: on Panda, CG-HIK loses four TRAC-IK-successful paths but completes two TRAC-IK failures; on UR5e it loses three and gains none. Thus the external comparison shows different coverage as well as a large implementation-level speed difference.

Geometric completion and timely completion also differ. Panda hard completes 37 trajectories geometrically but only 36 within 20 ms on every frame; full CG-HIK completes 34 under either definition. On UR5e the corresponding internal counts are all 36. TRAC-IK’s 36 Panda and 39 UR5e complete paths are also timely throughout. Frame-level timely acceptance is 95.95% / 95.75% for Panda/UR5e hard and 94.62% / 95.75% for full CG-HIK. The methods still process every fixed target; late calls are not truncated.

RQ3 is therefore not supported as a general successful-trajectory acceleration claim: the witnessed paths expose Panda completion losses and UR5e time overhead, alongside a real but localized joint-limit benefit. Combined with the original decomposition, these results locate the dominant aggregate savings in failure handling and motivate a more specific interpretation of learned allocation.

**Table 12: Predefined reference-trajectory families.** Each row has ten trajectories. Paired entries are hard / full CG-HIK. The last column reports TRAC-IK’s completion count and all-frame cumulative seconds. Full four-method family records and paired intervals accompany the source tables.

| Robot | Family             | Completed H / CG | Time H / CG (s) | FEV H / CG      | TRAC: completed / seconds |
|-------|--------------------|------------------|-----------------|-----------------|---------------------------|
| Panda | Smooth             | 9 / 9            | 31.998 / 5.195  | 32.758 / 5.445  | 9 / 0.938                 |
| Panda | Near-singular      | 8 / 5            | 18.124 / 28.070 | 24.291 / 35.387 | 7 / 1.375                 |
| Panda | Joint-limit return | 10 / 10          | 3.039 / 2.869   | 4.091 / 3.047   | 10 / 0.362                |
| Panda | High-curvature     | 10 / 10          | 2.186 / 2.869   | 2.996 / 2.996   | 10 / 0.364                |
| UR5e  | Smooth             | 8 / 8            | 6.124 / 5.221   | 9.162 / 6.761   | 10 / 0.362                |
| UR5e  | Near-singular      | 9 / 9            | 4.684 / 5.340   | 8.120 / 8.127   | 10 / 0.346                |
| UR5e  | Joint-limit return | 9 / 9            | 3.086 / 3.747   | 5.009 / 5.031   | 9 / 0.647                 |
| UR5e  | High-curvature     | 10 / 10          | 2.055 / 2.713   | 3.000 / 3.000   | 10 / 0.343                |



**Figure 5: Completion and all-frame cost on feasible reference paths.** Four methods begin from the same initial reference state and then evolve independently. The right panels include all 150 calls per path, including unsuccessful paths. TRAC-IK uses the budget selected on old calibration, not a point-test-selected winner.

## 7 Discussion

The supplementary evidence separates three questions often compressed into one runtime comparison: whether an entry avoids numerical work, whether its decision overhead is repaid, and whether accepted commands preserve later recoverability. The original aggregate gains are real under their recorded protocols, but their dominant source differs from the stronger successful-routing interpretation.

### 7.1 Allocation opportunity is not the same as learned advantage

Action-complete data are useful because they expose the cost of every available entry under a common command contract. They also expose when the allocation problem offers little headroom. Fixed hard is close to the empirical oracle on the matched population, and cross-fitting does not reveal a stable average improvement beyond it. A low router regret can therefore arise from near-equivalent options rather than accurate identification of a uniquely cheaper path.

The direct controls reinforce this reading. On feasible point inputs, full, geometry, and reject-only variants have similar cost, and all pay more than direct hard. The fixed cascade’s existing early stopping already makes numerical work query dependent; learned allocation must improve on that conditional execution, not receive credit for it. P95 selection produces a small FEV benefit over P50 selection, but the measured tail intervals do not establish its necessity. Predicting a tail cost remains a reasonable design objective whose incremental benefit must be demonstrated, rather than inferred from the objective itself.

### 7.2 Failure handling can dominate aggregate savings

Rejecting a request likely to exhaust the portfolio can avoid much more work than skipping a short numerical prefix. The old trajectory decomposition and the new full-versus-routing-only comparison both identify this mechanism. This is a useful form of resource allocation: no command is issued, and expensive unsuccessful work is avoided. It is distinct from accelerating a task that the solver completes.

The distinction is quantitative here. Original jointly completed paths take longer under CG-HIK, and the new jointly completed groups again pay additional wall time. Panda joint-limit-return shows that successful-solve numerical savings do occur, but the overall benefit is not uniform. Every-query candidate preparation and diagnostic inference impose a fixed cost that a small reduction in FEV may not repay. The correct comparison is the full returned-command pathway, with success and failure handling reported alongside its cumulative cost.

### 7.3 A verified current command need not preserve future recovery

The verifier certifies the current pose, joint, and velocity conditions. It does not certify that the resulting branch is favorable for later targets. The three lost Panda reference paths make this distinction observable: different valid first commands precede different later states, and the learned-entry paths eventually exhaust the same robust stages that still succeed from hard’s state. Disabling rejection does not restore those completions.

Development-only temporal shortcuts likewise produced long runs of inexpensive local solving but showed branch-dependent recovery and inconsistent cross-robot behavior. These observations motivate examining the whole closed-loop command sequence even for a policy that makes a fresh decision at each query. They are mechanism observations, not an additional main method or a new temporal experiment.

### 7.4 Practical position relative to an external solver

The TRAC-IK comparison changes the deployment interpretation. Under the tested exact-kinematics contract it generally returns feasible commands much faster than the learned-proposal portfolio, and its new reference-path completion exceeds full CG-HIK on both robots. CG-HIK retains a small UR5e point-success advantage and completes two Panda paths missed by TRAC-IK, so the solution sets are not identical. Those differences do not establish a general cost advantage.

This comparison includes implementation as well as numerical choices: native parallel TRAC-IK and the Python/PyTorch portfolio have different execution costs. For an application choosing between these measured systems, that cost is relevant. For an algorithmic attribution, the six shared-solver controls are the cleaner comparison. Taken together, they position CG-HIK as an interpretable study of learned portfolio allocation and failure handling, not as a demonstrated replacement for a mature general numerical IK solver.

## 8 Limitations

The command contract uses exact URDF forward kinematics and Jacobians. It excludes collision, contact, dynamics, torque, sensing, tracking, acceleration, and jerk. No physics simulator or physical robot is stepped. Zero accepted-contract violations mean consistency with these explicit kinematic conditions, not universal safety.

A feasible reference path establishes existence from its own sequence of states. Methods can leave that branch, so later frame inputs differ even under the same target index. Only the point study holds complete inputs fixed throughout. Outcome-conditioned trajectory groups describe observed cost and do not constitute randomized causal subsets.

The study uses two robot-specific models and predetermined supplementary families, not a prevalence-weighted deployment sample or cross-robot transfer test. Forty trajectories per robot and ten per family provide limited precision when a few paths dominate cumulative time. Paired bootstrap intervals preserve this trajectory dependence but do not establish distribution-free success guarantees. Five-repeat empirical P95 estimates are noisy; the independent-fold diagnostic reduces selection bias without turning them into a true optimal-cost oracle.

Latency depends on the recorded hardware, synchronization, and implementations. TRAC-IK uses Speed mode, its stated internal tolerance, and per-query limit updates; a different configuration is a different comparison. Some nonnegative TRAC-IK returns fail the common velocity check and remain failures: at the 5 ms point budget there are 157 such Panda calls and three UR5e calls, plus one UR5e search failure. The raw records retain those reasons. External FEV is not comparable. No method’s nominal command interval is treated as a measured real-time guarantee.

The frozen abstention rule also leaves many constructively inexecutable requests on the conservative defer path, where they still incur numerical work. The supplementary study does not retune that rule. Original OOD diagnostics and operational criteria are retained in Appendix A.4; no additional OOD experiment is introduced.

## 9 Conclusions

We studied whether learned query-conditioned costs can improve computation allocation within a fixed IK portfolio. CG-HIK supplies an action-complete learning formulation while preserving numerical command generation and deterministic acceptance.

The evidence distinguishes useful failure handling from a general learned-entry advantage. Original gains relative to fixed cascades remain intact, but their large trajectory savings mostly occur on paths neither method completes. Supplementary feasible-point controls do not outperform direct hard or a simple geometry rule; P95 selection yields small FEV savings without a resolved tail advantage. Continuously feasible paths retain a positive Panda joint-limit result, while Panda near-singular completion losses and UR5e total-time overhead prevent a cross-robot successful-path acceleration claim. TRAC-IK supplies a faster external operating point with different completion coverage.

The hypothesis that robust coverage need not require uniform worst-case computation therefore requires a more specific operational test than aggregate runtime alone. In this portfolio, learned abstention avoids substantial unsuccessful work, whereas learned entry selection does not establish the broader claimed benefit. The architectural separation remains: *learning allocates solver effort per query; numerical geometry generates joint commands; deterministic verification governs acceptance.*

## Data and Code Availability

The repository contains the manuscript source, figure source data, deterministic evidence extractor, frozen configurations, release manifests, and hash-addressed evaluation outputs. The generated evidence snapshot records the source path and SHA-256 for every value used in the paper. A citable archival DOI will be added to the accepted-manuscript record.

## Author Contributions

Author identities and CRediT roles are withheld for blinded review.

## Funding

Funding information is withheld for blinded review.

## Use of Generative Artificial Intelligence

OpenAI Codex assisted with supplementary experiment adapters, analysis code, language editing, LaTeX preparation, and reproducible figure generation. Original evidence is retained unchanged; supplementary tables and figures are derived from the recorded calls. The authors remain responsible for the scientific claims and for review and approval before submission.

## Conflicts of Interest

The authors declare no conflict of interest.

## A Original Frozen Evaluations

This appendix preserves the original evidence and its protocol boundaries. Its populations and profiling implementation differ from the supplementary study. No original result is replaced by a new measurement, and the original macros, source-data files, and solver configuration table remain unchanged.

### A.1 Original development diagnostics

Among successful action-complete development queries, the empirical oracle assignments were 26.69% easy, 27.77% medium, and 45.54% hard on Panda, and 23.41%, 25.36%, and 51.22% on UR5e. The original five-repeat in-sample best-fixed mean gaps were 0.137 and 0.154 ms; easy-entry gaps were 0.391 and 0.478 ms.

The separate successful, non-abstained policy-selection population had exact oracle agreement of 47.42% / 53.88%. Its mean regrets were 0.135 / 0.133 ms, medians 0.016 / 0.000 ms, and P95 0.467 / 0.605 ms. These original diagnostics describe different populations; they are not directly contrasted as a best-fixed-versus-learned improvement. RQ1 instead supplies the matched-population and independent-fold comparisons.

### A.2 Original point-query protocol and results

Each robot’s original point analysis has 14000 feasible queries: 10000 in-distribution and 4000 shifted feasible points, plus 2000 known-infeasible or one-frame-inadmissible points. Each feasible point has 3 raw calls per method. The primary released training seed is 17; 29 and 43 are sensitivity runs on the same queries, not independent replications.

**Table 13: Unchanged original feasible point results.** The primary comparator always enters the full cascade at easy. Latency is in milliseconds; values are read from the original frozen source-data export.

| Robot | Method               | Success (%) | Mean FEV | P50   | P95   | P99    | Fallback (%) |
|-------|----------------------|-------------|----------|-------|-------|--------|--------------|
| Panda | Fixed robust cascade | 99.03%      | 8.43     | 1.675 | 3.436 | 98.447 | 1.91%        |
| Panda | CG-HIK               | 99.03%      | 7.07     | 1.950 | 2.590 | 97.768 | 1.91%        |
| UR5e  | Fixed robust cascade | 100.00%     | 5.67     | 1.544 | 3.158 | 5.982  | 1.14%        |
| UR5e  | CG-HIK               | 100.00%     | 3.61     | 1.799 | 2.346 | 5.269  | 1.14%        |

Verified success was unchanged relative to the Fixed robust cascade. Mean FEV fell by 16.14% / 36.27%; P95 ratios were 0.7538 / 0.7427, and P99 ratios 0.9931 / 0.8808. Fallback was unchanged at 1.91% / 1.14%. On the constructed rejectable subset, command-reject recall was 95.65% / 93.95%, and rejected commands executed zero numerical stages and FEV. Relative to the fixed cascade, rejection avoided 95.69% / 93.92% of infeasible-point FEV.

### A.3 Original transition-rich trajectories

The original evaluation has 80 trajectories per robot, 20 in each of four families, with 150 frames at  $\Delta t = 20$  ms. The methods are the fixed easy-entry cascade, fixed hard-entry cascade, and CG-HIK. Each evolves its own accepted state while processing the common exogenous target sequence.

Aggregate time fell by 43.37% / 54.95%, and mean FEV by 45.16% / 65.12%. P50 increased by 11.66% / 11.09%; P95 fell by 12.04% / 21.94%. Panda P99 was effectively tied and UR5e P99 fell by 23.09%. Table 9’s decomposition identifies which completion groups produced these totals.

**Table 14: Unchanged original transition-rich results.** Cumulative latency sums every fixed-length frame. These original values are retained alongside, not replaced by, the witness-based supplementary evaluation.

| Robot | Method                   | Completed/80 | Time (s) | Mean FEV | P50   | P95     | P99     | Violations |
|-------|--------------------------|--------------|----------|----------|-------|---------|---------|------------|
| Panda | Fixed robust cascade     | 41/80        | 487.64   | 63.79    | 1.653 | 236.316 | 355.565 | 0          |
| Panda | Fixed hard-entry cascade | 40/80        | 472.04   | 60.85    | 1.651 | 229.142 | 353.570 | 0          |
| Panda | CG-HIK                   | 41/80        | 267.31   | 33.37    | 1.844 | 201.564 | 353.672 | 0          |
| UR5e  | Fixed robust cascade     | 34/80        | 143.30   | 39.15    | 1.556 | 38.809  | 51.463  | 0          |
| UR5e  | Fixed hard-entry cascade | 34/80        | 138.23   | 35.23    | 1.558 | 38.104  | 50.821  | 0          |
| UR5e  | CG-HIK                   | 34/80        | 62.27    | 12.29    | 1.731 | 29.745  | 39.085  | 0          |

Original family cumulative-time reductions were 59.26% / 59.60% for near-singular, 44.09% / 20.47% for high-curvature/high-speed, and 29.62% / 9.44% for smooth-orientation transitions. Panda joint-limit reductions were -0.68% in time and -0.58% in FEV, indicating regressions where negative. UR5e’s joint-limit saving accompanied zero completions under both methods. Panda high-curvature CG-HIK completion remained 1/20. These family totals include unsuccessful trajectories and do not imply the same effects on jointly completed paths.

#### A.4 Original protocol boundaries

The original primary point system uses training seed 17. The two additional training seeds reuse the same queries. Prespecified point margins and a legacy trajectory-completion margin per robot form one joint Holm family; all margin nulls were rejected. That legacy trajectory arm is distinct from the original final transition-rich evaluation. A separate operational gate required a strictly positive improvement in OOD-feasible false rejection. Panda fixed and CG-HIK both had zero such false rejects, so that condition and the original point-study paper gate were false. No threshold was changed after observing this result.

The original transition-rich criteria required completion not below hard, total-time and FEV ratios at most 0.85 and 0.85, P95 at most 1.00, P99 at most 1.05, and zero contract violations. P50 was report-only. These criteria are historical protocol facts, not a new composite gate applied to the supplementary data.

Original OOD AUROC was 0.430 / 0.504 and AUPRC 0.210 / 0.230, with no successful defer recovery (0.00% / 0.00%). The original categorical and Cartesian-step development baselines are distinct from the new controlled geometry rule that shares frozen abstention.

#### A.5 Reproducibility without further experiments

The original evidence builder and figure source data are retained unchanged. The supplementary paper exporter reads only completed report tables, verifies figure hashes, and writes a separate source-data and figure directory. Rebuilding this manuscript executes no IK call and changes no model or output record. The frozen predictor, normalization, calibrators, OOD transform, and policy constants remain exported as one batch-one runtime module [38]. The new findings report, method mapping, raw records, complete UID sets, and measurement manifests accompany the manuscript.

## References

- [1] Daniel Rakita, Bilge Mutlu, and Michael Gleicher. RelaxedIK: Real-time synthesis of accurate and feasible robot arm motion. In *Robotics: Science and Systems XIV*, 2018. doi: 10.15607/RSS.2018.XIV.043.
- [2] Jeremy Morgan, David Millard, and Gaurav S. Sukhatme. CppFlow: Generative inverse kinematics for efficient and robust cartesian path planning. In *2024 IEEE International Conference on Robotics and Automation*, 2024. doi: 10.1109/ICRA57147.2024.10611724.
- [3] Duc Tien Nguyen, Van Thanh Tri Nguyen, Duc Toan Luu, Truong Do, and Vu Linh Nguyen. Trajectory-based sequential learning for efficient and stable inverse kinematics of robotic manipulators. *Robotics and Autonomous Systems*, 205: 105669, 2026. doi: 10.1016/j.robot.2026.105669. In press as of 2 September 2026; scheduled for the November 2026 issue.
- [4] Jiahao Yang, Shenhao Yan, Fan Feng, Chengsi Yao, Ge Wang, Zhixin Mai, Yiming Zhao, and Yatong Han. MimicIK: Real-time generative inverse kinematics from teleoperation with fk consistency, 2026. Preprint.
- [5] Alexander J. Elias and John T. Wen. IK-Geo: Unified robot inverse kinematics using subproblem decomposition. *Mechanism and Machine Theory*, 209:105971, 2025. doi: 10.1016/j.mechmachtheory.2025.105971.
- [6] Daniel Ostermeier, Jonathan Külz, and Matthias Althoff. Automatic geometric decomposition for analytical inverse kinematics. *IEEE Robotics and Automation Letters*, 10(10):9964–9971, 2025. doi: 10.1109/LRA.2025.3597897.

- [7] Pablo C. Lopez-Custodio, Yuhe Gong, and Luis F. C. Figueredo. GeoFIK: A fast and reliable geometric solver for the IK of the Franka arm based on screw theory enabling multiple redundancy parameters, 2025. URL <https://arxiv.org/abs/2503.03992>. Preprint; first posted in 2025, latest revision in 2026.
- [8] Charles W. Wampler, II. Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods. *IEEE Transactions on Systems, Man, and Cybernetics*, 16(1):93–101, 1986. doi: 10.1109/TSMC.1986.289285.
- [9] Yoshihiko Nakamura and Hideo Hanafusa. Inverse kinematic solutions with singularity robustness for robot manipulator control. *Journal of Dynamic Systems, Measurement, and Control*, 108(3):163–171, 1986. doi: 10.1115/1.3143764.
- [10] Stefano Chiaverini, Bruno Siciliano, and Olav Egeland. Review of the damped least-squares inverse kinematics with experiments on an industrial robot manipulator. *IEEE Transactions on Control Systems Technology*, 2(2):123–134, 1994. doi: 10.1109/87.294335.
- [11] Patrick Beeson and Barrett Ames. TRAC-IK: An open-source library for improved solving of generic inverse kinematics. In *2015 IEEE-RAS 15th International Conference on Humanoid Robots*, pages 928–935, 2015. doi: 10.1109/HUMANOIDS.2015.7363472.
- [12] Jacinto Colan, Ana Davila, and Yasuhisa Hasegawa. Variable step sizes for iterative jacobian-based inverse kinematics of robotic manipulators. *IEEE Access*, 12:87909–87922, 2024. doi: 10.1109/ACCESS.2024.3418206.
- [13] Liangting Wu and Roberto Tron. IKSPARK: Obstacle-aware inverse kinematics via convex optimization, 2024. URL <https://arxiv.org/abs/2403.12235>. Preprint; first posted in 2024, latest revision in 2026.
- [14] Yachen Zhang and Ryo Kikuuwe. Ensuring viability: A QP-based inverse kinematics for handling joint range, velocity and acceleration limits, as well as whole-body collision avoidance. *Journal of Intelligent & Robotic Systems*, 112(1):16, 2026. doi: 10.1007/s10846-025-02335-z.
- [15] Cael Yasutake, Andrew H. Liu, Zachary Kingston, and Brian Plancher. HJCD-IK: Gpu-accelerated inverse kinematics through batched hybrid jacobian coordinate descent, 2025. URL <https://arxiv.org/abs/2510.07514>. Preprint; first posted in 2025, revised in 2026; accepted to the 2026 IEEE/RSJ International Conference on Intelligent Robots and Systems.
- [16] Bruno Siciliano. Kinematic control of redundant robot manipulators: A tutorial. *Journal of Intelligent and Robotic Systems*, 3(3):201–212, 1990. doi: 10.1007/BF00126069.
- [17] Barrett Ames, Jeremy Morgan, and George Konidaris. IKFlow: Generating diverse inverse kinematics solutions. *IEEE Robotics and Automation Letters*, 7(3):7177–7184, 2022. doi: 10.1109/LRA.2022.3181374.
- [18] Oliver Limoyo, Filip Marić, Matthew Giamou, Petra Alexson, Ivan Petrović, and Jonathan Kelly. Generative graphical inverse kinematics. *IEEE Transactions on Robotics*, 41:1002–1018, 2025. doi: 10.1109/TRO.2024.3521862.
- [19] Mochammad Rizky Diprasetya, Johannes Pöppelbaum, and Andreas Schwung. KineNN: Kinematic neural network for inverse model policy based on homogeneous transformation matrix and dual quaternion. *Robotics and Computer-Integrated Manufacturing*, 94:102945, 2025. doi: 10.1016/j.rcim.2024.102945.
- [20] Youn-Jae Go and Jun Moon. EMIKNet: Expanding multiple-instance inverse kinematics network for multiple end-effector and multiple solutions. *IEEE Access*, 13:25087–25096, 2025. doi: 10.1109/ACCESS.2025.3539022.
- [21] Zeyu Zhang and Ziyuan Jiao. IKDiffuser: A diffusion-based generative inverse kinematics solver for kinematic trees, 2025. URL <https://arxiv.org/abs/2506.13087>. Preprint; first posted in 2025, revised in 2026; under review.
- [22] Duc Tien Nguyen, Van Thanh Tri Nguyen, Truong Do, and Vu Linh Nguyen. Multi-solution inverse kinematics for robotic manipulators via permutation-invariant set prediction. *IEEE Robotics and Automation Letters*, 11(8):9803–9810, 2026. doi: 10.1109/LRA.2026.3703280.
- [23] Jacket Demby’s, Ramy Farag, and Guilherme N. DeSouza. Inverse kinematics of robotic manipulators using a new learning-by-example method. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 9534–9541, 2024. doi: 10.1109/IROS58592.2024.10802048.
- [24] Jan-Gerrit Habekost, Erik Strahl, Philipp Allgeuer, Matthias Kerzel, and Stefan Wermter. CycleIK: Neuro-inspired inverse kinematics. In *Artificial Neural Networks and Machine Learning – ICANN 2023*, volume 14254 of *Lecture Notes in Computer Science*, pages 457–470. Springer, 2023. doi: 10.1007/978-3-031-44207-0\_38.
- [25] Huang Huang, Balakumar Sundaralingam, Arsalan Mousavian, Adithyavairavan Murali, Ken Goldberg, and Dieter Fox. DiffusionSeeder: Seeding motion optimization with diffusion for rapid motion planning. In Pulkit Agrawal, Oliver Kroemer, and Wolfram Burgard, editors, *Proceedings of the 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 4392–4409. PMLR, 2025. URL <https://proceedings.mlr.press/v270/huang25f.html>.

- [26] João Carvalho, An Thai Le, Piotr Kicki, Dorothea Koert, and Jan Peters. Motion planning diffusion: Learning and adapting robot motion planning with diffusion models. *IEEE Transactions on Robotics*, 41:4881–4901, 2025. doi: 10.1109/TRO.2025.3593109.
- [27] Yucheng Tang, Xi Huang, Yongzhou Zhang, Tao Chen, Ilshat Mamaev, and Björn Hein. ETA-IK: Execution-time-aware inverse kinematics for dual-arm systems. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 7504–7511, 2025. doi: 10.1109/IROS60139.2025.11247583.
- [28] Meenalochani Jayabalan, Karunamoorthy Loganathan, and Palanikumar Kayaroganam. A novel hybrid ik architecture for robotic arms: Iterative refinement of soft-computing approximations with validation on abb irb-1200 robotic arm. *Machines*, 14(3):292, 2026. doi: 10.3390/machines14030292.
- [29] Ali Jlidi, Rabab Benotsmane, and László Kovács. Explainable graph neural networks towards data-driven inverse kinematics in industrial robot motion planning. *Electronics*, 15(14):3071, 2026. doi: 10.3390/electronics15143071.
- [30] Xinyi Yuan, Weiwei Wan, and Kensuke Harada. IKSel: Selecting good seed joint values for fast numerical inverse kinematics iterations. *IEEE Transactions on Automation Science and Engineering*, 23:4410–4427, 2026. doi: 10.1109/TASE.2026.3659225.
- [31] Chen Xu, Tony Khuong Nguyen, Emma Dixon, Christopher Rodriguez, Patrick Miller, Robert Lee, Paarth Shah, Rares Andrei Ambrus, Haruki Nishimura, and Masha Itkina. Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies. In *Proceedings of Robotics: Science and Systems XXI*, Los Angeles, CA, USA, 2025. doi: 10.15607/RSS.2025.XXI.073.
- [32] John R. Rice. The algorithm selection problem. In Morris Rubinoff and Marshall C. Yovits, editors, *Advances in Computers*, volume 15, pages 65–118. Academic Press, 1976. doi: 10.1016/S0065-2458(08)60520-3.
- [33] Bernd Bischl, Pascal Kerschke, Lars Kotthoff, Marius Lindauer, Yuri Malitsky, Alexandre Fréchet, Holger Hoos, Frank Hutter, Kevin Leyton-Brown, Kevin Tierney, and Joaquin Vanschoren. ASlib: A benchmark library for algorithm selection. *Artificial Intelligence*, 237:41–58, 2016. doi: 10.1016/j.artint.2016.04.003.
- [34] Ivan Boschi, Alessandro De Toni, Roberto Di Leva, Edoardo Idà, and Marco Carricato. Handling transitions across singularities for UR-like serial robots. *IEEE Robotics and Automation Letters*, 11(3):2714–2721, 2026. doi: 10.1109/LRA.2026.3653295.
- [35] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pages 1321–1330. PMLR, 2017. URL <https://proceedings.mlr.press/v70/guo17a.html>.
- [36] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL [https://proceedings.neurips.cc/paper\\_files/paper/2017/hash/9ef2ed4b7fd2c810847ffa5fa85bce38-Abstract.html](https://proceedings.neurips.cc/paper_files/paper/2017/hash/9ef2ed4b7fd2c810847ffa5fa85bce38-Abstract.html).
- [37] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- [38] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL [https://proceedings.neurips.cc/paper\\_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html](https://proceedings.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html).
- [39] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011. URL <https://www.jmlr.org/papers/v12/pedregosa11a.html>.